RedByte

FPGA EDUCATIONAL IDE

Product Manual

Version 1.0 · March 2026

Target Platform: Digilent Basys 3 (Artix-7 XC7A35T)

Toolchain: AMD Vivado ML Standard Edition

DOCUMENT CONTROL

Document ID	RB-MAN-001
Version	1.0
Date	2026-03-31
Classification	Product Reference
Repository	`redbyte-ui` monorepo
Primary Package	`packages/rb-apps`
Target Board	Digilent Basys 3 (Artix-7 XC7A35T)
Part Number	`xc7a35t-1cpg236-1`
License	RedByte Proprietary License (RPL-1.0)

CONVENTIONS

NOTE

Supplementary information or clarification.

WARNING

Important information that may affect results.

CAUTION

Action that may cause data loss or errors.

TIP

Best practice or recommended approach.

Table of Contents

D. Import Fidelity Reference

1. Introduction

This manual is the canonical product reference for RedByte, a browser-based digital logic design, simulation, verification, and FPGA hardware-preparation environment. It describes the product as it exists, explains its operating model, documents each major surface and workflow, and provides reference material for all audiences.

Students beginning their first project should read Sections 2 through 6, then follow the workflow sections (8 through 11) in order. **Instructors and graders** should additionally read Section 13 for submission and review workflows. **Evaluators and developers** will find the appendices and glossary useful as standing reference material.

2. Product Overview

2.1 What RedByte Is

RedByte is an interactive digital logic circuit design and verification environment that runs entirely in a web browser. It provides a unified workspace where users design combinational and sequential circuits from primitive logic gates, verify their behavior against test vectors, map circuit ports to physical FPGA board pins, and export a complete file set for synthesis and programming in AMD Vivado.

RedByte targets the **Digilent Basys 3** development board (Xilinx Artix-7 XC7A35T FPGA). It replaces the fragmented workflow of separate schematic editors, simulation tools, and manual constraint-file authoring with a single integrated environment.

2.2 Core Capabilities

CAPABILITY	DESCRIPTION
Circuit Design	Visual drag-and-drop canvas for constructing digital circuits from 25+ logic primitives including gates, flip-flops, and timing elements.
Deterministic Simulation	Tick-based engine using topological-sort evaluation. Same circuit + same inputs = same outputs, every time, on every machine.
Truth Table Verification	Automated testing against expected truth table vectors with per-row pass/fail reporting. Supports combinational and sequential schedules.
Hardware Pin Mapping	Dedicated interface for assigning circuit ports to Basys 3 physical pins — switches, LEDs, buttons, 7-segment display, clock.
Vivado Export	Complete export pipeline generating synthesizable VHDL, pin constraints (XDC), and a VHDL testbench in a single ZIP.
Project Import	Import VHDL, XDC, or RedByte project archives with explicit fidelity reporting (Full, Reconstructed, or Partial).
Submission Packaging	Deterministic, tamper-evident submission export with SHA-256 integrity hashing for instructor review.

2.3 Design Philosophy

PRINCIPLE	MEANING
Determinism by Design	Every simulation tick is reproducible. No hidden randomness, no race conditions, no undefined behavior.
One Truth, Many Views	The circuit is the single source of truth. Every surface is a projection of the same data.
Truth over Simplification	Gates have propagation delay. Circuits stabilize over ticks. Students learn correct mental models.
Local-First Operation	All computation happens in the browser. No server, no account, no data leaves the machine.

3. Intended Users and Usage Contexts

AUDIENCE	PRIMARY CONTEXT	KEY ACTIVITIES
Students	IdeApp, LabWorkspaceApp	Design circuits, verify truth tables, map pins, export for Vivado, submit lab work.
Instructors / TAs	Submission Inspector (via Project surface)	Review submissions, verify integrity, inspect gate verdicts, grade with full diagnostics.
Self-Learners	LogicPlaygroundApp	Open-ended circuit exploration with no submission pipeline.
Evaluators	All contexts	Assess product capabilities, review workflows, evaluate platform for adoption.

Application Contexts

CONTEXT	PURPOSE	SUBMISSION SYSTEM	DIAGNOSTIC DETAIL
IdeApp	Full six-surface IDE for lab assignments	Yes	Student-appropriate (simplified)
LogicPlaygroundApp	Open-ended sandbox	No	Minimal
LabWorkspaceApp	Guided lab with step tracking	Yes	Student-appropriate
SubmissionInspectorApp	Instructor grading tool (inspector functionality delivered via Project surface)	Read-only review	Full (hashes, verdicts, tamper detection)

4. Core Concepts and Operating Model

4.1 The Canonical Workflow

RedByte organizes work into a linear progression of six surfaces. A sixth surface, **Import**, is available at any time for bringing external HDL or previously exported projects into the environment. Users may navigate between surfaces freely; the system tracks completion status for each stage.

4.2 Simulation Model

RedByte's simulation engine operates on discrete **ticks**. Each tick, the engine evaluates every node in **topological order** — nodes whose inputs depend only on already-evaluated nodes are evaluated first. This ordering guarantees deterministic, race-free signal propagation.

> **NOTE**
>
> For **combinational circuits**, evaluation typically stabilizes within one or two ticks. For **sequential circuits**, the engine detects clock edges by comparing the current clock value against the previous tick's value. A rising edge (0→1) triggers data capture in flip-flops.

4.3 Verification Model

Verification compares actual circuit outputs against expected values defined in **test vectors**. RedByte automatically selects the correct verification schedule:

SCHEDULE	CIRCUIT TYPE	BEHAVIOR PER VECTOR ROW
Combinational	No flip-flops	Apply inputs → one tick → read outputs
Clocked Macro	Contains flip-flops	Apply inputs → CLK=0 tick → CLK=1 tick → CLK=0 tick → read outputs

4.4 Architecture Layers

Layer E — UX Shell	Student-facing surfaces (IdeApp.tsx + 6 surfaces). No internal diagnostic language in default view.
Layer D — Submission	Deterministic ZIP bundling with integrity hashes. Tamper detection and gate verdicts.
Layer C — Vivado Adapter	VHDL generation, XDC constraints, testbench generation, sequential analysis.
Layer B — Verification	Truth table comparison, pass/fail per vector, failing node identification.
Layer A — Logic Core	Deterministic circuit simulation. Gate evaluation, topological sort, signal propagation.

Each layer depends only on layers below it. The Logic Core has no upward dependencies. The UX Shell orchestrates calls to all lower layers on user actions.

4.5 Import Fidelity Model

LEVEL	NAME	SOURCE	WHAT IS PRESERVED
1	Full	RedByte project archive (`.rbproj.json`)	All state: circuit, layout, vectors, probes, mappings.
2	Reconstructed	Structural VHDL (component instantiation)	Topology correct. Auto-layout. No vectors.
3	Partial	Behavioral VHDL / concurrent assignments	I/O port names and directions only.

5. Getting Started

5.1 System Requirements

RedByte runs in any modern web browser with JavaScript enabled (Chrome, Edge, or Firefox recommended). For FPGA synthesis, **AMD Vivado ML Standard Edition** (2024.1+) and a USB connection to the Basys 3 board are required.

5.2 Development Installation

```
git clone <repository-url>
cd redbyte-ui
pnpm install
pnpm dev
```

The development server starts at `http://localhost:5173`.

5.3 Starter Examples

EXAMPLE	DESCRIPTION	TYPE
Signal Tour	Four-wire passthrough (SW→LD)	Combinational
Logic Gates	AND, OR, XOR with 2 switches, 3 LEDs	Combinational
Half Adder	Sum and carry from two inputs	Combinational
Full Adder	1-bit addition with carry-in/out	Combinational
Two-Bit Counter	2-bit counter with clock and reset	Sequential

5.4 Quick Walkthrough: AND Gate to Vivado Export

1. Open RedByte and navigate to the **Design** surface.
2. Drag two **Switch** nodes and one **AND** gate from the palette onto the canvas.
3. Drag one **Lamp** node onto the canvas.
4. Wire each Switch output to an AND gate input. Wire the AND output to the Lamp input.
5. Navigate to **Verify**. Add truth table vectors or use defaults. Click **Run Verify**.
6. Navigate to **Hardware**. Assign switches to SW0/SW1 and the lamp to LD0.
7. Navigate to **Export**. Click **Download Vivado Kit**.

> **Result:** A ZIP file containing `top.vhd`, `top.xdc`, `testbench.vhd`, automation scripts, and documentation files, ready for Vivado import. See Section 11.1 for the complete file list.

6. Workspace and Interface Overview

6.1 Global Shell

The IDE shell consists of four persistent regions visible on every surface:

REGION	LOCATION	CONTENT
Top Bar	Top	Product mark, project name, save state, contextual actions (Run Verify, Export, Help).
Left Rail	Left	Six mode entries: Project, Design, Verify, Hardware, Export, Import. Active marker and progress indicators.
Main Content	Center	Active surface content. Changes entirely when switching surfaces.
Status Bar	Bottom	Application version, last verification status.

6.2 Navigation and Status

Surface switching is always explicit — the user clicks a mode entry in the left rail. The system does not auto-switch surfaces. Each surface displays a maximum of **three status pills** (circuit health, verify result, export readiness) using deterministic language: PASS, FAIL, READY, BLOCKED.

7. Detailed Surface Reference

7.1 Project Surface

ASPECT	DETAIL
Purpose	Project-level overview, metadata management, starter example selection, submission export.
When to Use	Session start, project creation, readiness review, metadata changes.
Main Area	Summary cards: project identity, Basys 3 target, last verification, IO mapping completeness.
Controls	Edit name/description. Open starter example (with overwrite guard). Review readiness.
Outputs	Updated metadata. Readiness assessment. Submission export.
Student View	Lab name, student name input, verify status, "Export Submission" button. Hash data in Advanced panel only.

7.2 Design Surface

ASPECT	DETAIL
Purpose	Circuit construction via drag-and-drop gate placement and wiring.
When to Use	Whenever the circuit needs to be created or modified.
Main Area	Circuit canvas with tool row (select, wire, delete, zoom). Left: searchable component palette. Right: selected element properties.
Controls	Place components, wire ports, select/move/delete, undo/redo (50 levels).
Wiring Rules	Output→Input valid. Fan-out allowed. Self-loops rejected. One connection per input port.
Health Indicator	"Circuit OK" or "Issues found" (floating outputs, missing I/O).

TIP

When navigating to Design from a failing verify result, the system highlights the relevant gate and displays a diagnostic callout.

7.3 Verify Surface

ASPECT	DETAIL
Purpose	Run truth table verification, present pass/fail verdicts, enable failure analysis.
When to Use	After building or modifying a circuit, before hardware mapping or export.
Main Area	Top: PASS/FAIL banner with run summary. Center: truth table with per-row results. Right: signal picker and waveform preview.
Controls	Run Verify, Jump to failing node, Inspect failure diffs, Edit vectors, Scenario Builder.
Schedule	Auto-detected: combinational (1 tick) or clocked-macro (3-tick clock sequence) based on flip-flop presence.
Hints	14 diagnostic conditions evaluated on FAIL; matching fact-grounded hints displayed. Sequential circuit banner when DFF detected.
Freshness	Stale indicator when circuit modified since last verify. Name/description changes do not stale.

7.4 Hardware Surface

ASPECT	DETAIL
Purpose	Map circuit ports to physical Basys 3 pins. Required before export.
When to Use	After circuit is designed and verified.
Tabs	Map Pins, Prepare Board, Program Checklist, Live Details. Tabs appear above the hero card.
Controls	Pin assignment dropdowns (filtered by direction), pin presets, clear mapping.
Callout Strip	Compare and Export status always visible regardless of active tab.
Available Pins	16 switches, 16 LEDs, 5 buttons, 7-segment display (7 cathodes + 4 anodes + DP), CLK100MHZ.

7.5 Export Surface

ASPECT	DETAIL
Purpose	Validate readiness, preview generated artifacts, produce Vivado Kit ZIP.
When to Use	After design, verification, and complete pin mapping.
Main Area	Top: READY/WARNING/BLOCKED status. Center: artifact preview for primary files (`top.vhd`, `top.xdc`, `testbench.vhd`). Right: pin table, validation.
Primary CTA	**Download Vivado Kit** — downloads ZIP. Enabled only at READY status.
Single Path	One generation path. Preview shows exact bytes that appear in the ZIP.
Scaffold Warnings	HDL/XDC port mismatch warnings for projection-only exports are informational, not blocking.

7.6 Import Surface

ASPECT	DETAIL
Purpose	Import external HDL, constraint files, or RedByte project archives.
Tabs	"Upload ZIP" (file picker), "Paste HDL" (paste VHDL source), "Paste XDC" (constraint text).
Main Area	Parsed ports table, schematic preview, diagnostics list with plain-language errors.
Fidelity	Full (project archive), Reconstructed (structural VHDL), Partial (behavioral VHDL). Reported explicitly.
Submission Detection	Identifies RedByte submission ZIPs and reports integrity status.

WARNING

RedByte's own exported `top.vhd` uses concurrent signal assignments optimized for synthesis. It imports at **Partial** fidelity only. For full-fidelity re-import, use the `.rbproj.json` file from the same export ZIP.

8. Circuit Design Workflow

8.1 Placing Components

1. Open the **Design** surface.
2. Locate the desired component in the left palette (use search to filter by name).
3. Drag the component onto the canvas and release to place.
4. Repeat for all required components.

8.2 Wiring Components

1. Click on an **output port** (right side of a gate, or Switch output).
2. Drag to the target **input port** (left side of a gate, or Lamp input).
3. Release on the target port.

> **Result:** A wire appears. The signal flows from source to destination. Invalid connections are rejected with feedback.

8.3 Sequential Elements

When building circuits with flip-flops:

1. Place the flip-flop (e.g., D Flip-Flop) on the canvas.
2. Connect the **D** (data) port to the source signal.
3. Connect the **CLK** port to a Clock node or designated clock switch.
4. Optionally connect **EN** (enable) and **RST** (reset) ports.

5 Connect the **Q** output to downstream logic or output indicators.

> **NOTE**
>
> The D flip-flop captures input on the **rising edge** (0→1 transition) of the clock. Between edges, output Q holds its value. The complementary output $\bar{Q}$ is also available.

9. Verification Workflow

1. Navigate to the **Verify** surface.
2. Review or author test vectors in the truth table. Each row specifies inputs and expected outputs.
3. Click **Run Verify**.
4. Review results: ✓ per passing row, × per failing row.
5. For failures: use **Jump to failing node** to navigate to the Design surface with the relevant gate highlighted.
6. Fix the circuit and re-verify until all vectors pass.

> **Result:** A PASS verdict confirms the circuit is correct for all tested input combinations. A FAIL verdict identifies specific rows and signals that do not match expectations.

Verification Determinism

Results are deterministic. Running the same circuit with the same vectors produces the same results every time, on every machine. The Verify surface displays a deterministic run hash to confirm reproducibility.

10. Hardware Mapping and Board Preparation

1. Navigate to the **Hardware** surface, **Map Pins** tab.
2. For each circuit port, select a Basys 3 pin from the dropdown. Inputs map to switches/buttons; outputs map to LEDs/display segments.
3. For sequential circuits, assign the clock port to **CLK100MHZ** (pin W5).
4. Confirm "Mapping complete" status.

> **NOTE**
>
> The export pipeline automatically emits `CLOCK_BUFFER_TYPE NONE` for switch and button inputs. This prevents Vivado from inserting illegal BUFG paths on non-clock-capable pins.

11. Vivado Export and External Tool Workflow

11.1 Generated Files

FILE	TYPE	PURPOSE
`top.vhd`	Design Source	Synthesizable VHDL top-level entity.
`top.xdc`	Constraints	Pin assignments with LVCMOS33 I/O standard.
`testbench.vhd`	Simulation Source	VHDL testbench from truth table vectors.
`vivado_import.tcl`	Automation	Tcl script for automated Vivado project creation.
`program_and_test.tcl`	Automation	Tcl script for board programming and test.
`README.txt`	Documentation	Quick-start instructions for the Vivado workflow.
`BRINGUP.md`	Documentation	Detailed board bring-up guide.
`EXPECTED_IO.json`	Metadata	Machine-readable I/O expectations for automated verification.
`project.rbproj.json`	Project	RedByte project snapshot for round-trip import.

11.2 Importing into Vivado

1. **Download the Vivado Kit** from the Export surface and extract the ZIP.
2. Open Vivado → **Create Project** → RTL Project.
3. Add `top.vhd` as Design Source. Add `top.xdc` as Constraints.
4. Select part: `xc7a35t-1cpg236-1` (or board "Basys3").
5. **Run Synthesis** → **Run Implementation** → **Generate Bitstream**.
6. Open Hardware Manager → connect board → **Program Device** with the generated `.bit` file.

> **Result:** The FPGA is programmed. Toggle physical switches and observe LEDs to verify the design matches simulation.

11.3 Common Vivado Errors

ERROR	LIKELY CAUSE	RESOLUTION
"Port X not found in entity"	Port name mismatch	Re-map in Hardware surface and re-export.
"Multiple drivers on net"	Combinational loop	Fix the loop in Design, re-verify, re-export.
"Timing not met"	Excessive logic depth	Simplify combinational path or add pipeline registers.

12. Import and Reuse Workflows

12.1 Full-Fidelity Re-import

Upload the `.rbproj.zip` archive on the Import surface's "Upload ZIP" tab. The system detects the project manifest and restores all state.

12.2 Structural VHDL Import

Paste structural VHDL with component instantiation on the "Paste HDL" tab. The parser maps 37 HDL name variants (e.g., `and2`, `AND`, `and_gate`) to 9 RedByte node types through a built-in component library. Topology is reconstructed; positions are auto-assigned; test vectors must be re-authored.

12.3 XDC Constraint Import

Paste constraint file content on the "Paste XDC" tab. Pin assignments are imported and applied to matching circuit ports.

13. Submission, Review, and Instructor Workflows

13.1 Student Submission

1. Complete circuit design and achieve a `PASS` verification.
2. Navigate to the **Project** surface.
3. Click **Export Submission**.
4. Upload the downloaded archive to the institutional LMS.

The submission is **deterministic**: same project state always produces the same bytes. This enables tamper detection.

13.2 Instructor Review

1. Open the **Submission Inspector**.
2. Import the student's submission archive.
3. Review: gate verdict, pass/fail per requirement, integrity status, hash verification, tamper detection.
4. Click **Export Grading Report** to download a JSON summary.

13.3 Integrity Properties

PROPERTY	MECHANISM
Deterministic packaging	Same state → identical output bytes.
Content-addressed hashing	SHA-256 hash per file in manifest.
Tamper detection	Any file modification invalidates checksums.
Optional signing	Ed25519 signatures. Unsigned accepted but marked.

13.4 Gate Verdicts

CONTEXT	DISPLAY
Student view (IdeApp)	"Ready to submit" / "Submission needs attention"
Instructor view (Inspector)	Raw PASS/FAIL with detailed breakdown

14. Files, Packages, and Generated Outputs

14.1 Vivado Kit ZIP

```
vivado-kit.zip
├── top.vhd                 # Synthesizable VHDL
├── top.xdc                 # Basys 3 pin constraints
├── testbench.vhd           # Simulation testbench
├── vivado_import.tcl       # Automated Vivado project creation
├── program_and_test.tcl    # Board programming and test script
├── README.txt              # Quick-start instructions
├── BRINGUP.md              # Board bring-up guide
├── EXPECTED_IO.json        # Machine-readable I/O expectations
└── project.rbproj.json     # RedByte project snapshot
```

14.2 Submission Package

```
submission.rbproj.zip
├── rb-project.json         # Complete project state
├── verify-ledger.json      # Verification results
├── manifest.json           # File hashes / integrity
└── [metadata files]        # Gate verdicts, run counts
```

14.3 RB Lab ZIP (Bridge MVP)

```
<lab>.rb-lab.zip
├── manifest.json
├── trace/hw_trace.ndjson
├── bitstream/design.bit
├── meta/board_profile.json
└── integrity/
    ├── capsule.json
    └── signature.sig
```

15. Troubleshooting and Error Resolution

15.1 Circuit Design Issues

SYMPTOM	CAUSE	RESOLUTION
Wire will not connect	Invalid direction	Source must be output port, target must be input port.
"Issues found" health	Floating outputs / missing I/O	Connect all outputs; add INPUT/OUTPUT nodes for hardware ports.
Flip-flop output unchanged	No clock edge	Ensure Clock node connected to CLK port and value transitions 0→1.

15.2 Verification Issues

SYMPTOM	CAUSE	RESOLUTION
All vectors fail	Wiring error	Trace signal paths in Design surface.
Sequential test fails	Missing clock	Connect clock to all flip-flops.
Stale indicator	Circuit modified	Re-run verification.

15.3 Export Issues

SYMPTOM	CAUSE	RESOLUTION
Export BLOCKED	Incomplete prerequisites	Complete design, verify, and pin mapping.
"HDL ports missing in XDC"	Scaffold warning	Expected for simple circuits. Does not block export.
Port name error in VHDL	Invalid characters	Use lowercase letters and underscores only in port labels.

15.4 Vivado Issues

SYMPTOM	CAUSE	RESOLUTION
Synthesis: "Port not found"	Name mismatch	Re-export from RedByte.
Routing errors	Complex logic / loops	Simplify circuit; check for combinational loops.
Board unresponsive after program	Wrong FPGA part	Verify Vivado project targets `xc7a35t-1cpg236-1`.

16. Best Practices

DESIGN

Begin with INPUT and OUTPUT nodes to define the circuit boundary before adding internal logic. Use meaningful labels — they propagate to exported HDL port names.

VERIFICATION

Define test vectors before building complex circuits. Test edge cases: all-zero, all-one, and boundary conditions. Re-verify after every design change.

HARDWARE

Assign clock ports first for sequential designs. Use pin presets for standard lab configurations.

EXPORT

Use `.rbproj.json` for re-import, not `top.vhd`. Keep the exported ZIP alongside the Vivado project as a design record.

SUBMISSION

Fill in the student name field before exporting. Do not manually modify files inside the ZIP — any change invalidates integrity hashes.

17. Glossary

TERM	DEFINITION
Basys 3	Digilent FPGA development board (Artix-7 XC7A35T). The target hardware platform for RedByte exports.
Circuit	A collection of nodes and connections representing a digital logic design. The single source of truth.
Clocked-macro schedule	Verification schedule for sequential circuits. Three-tick clock sequence per vector row.
Connection	Directional link from output port to input port. Format: `{ from: { nodeId, portName }, to: { nodeId, portName } }`.
Determinism	Identical inputs always produce identical results, across runs and machines.
Fidelity level	Import quality classification: Full, Reconstructed, or Partial.
Gate verdict	Summary judgment of whether work meets lab requirements.
LVCMOS33	3.3V low-voltage CMOS I/O standard used by the Basys 3 board.
Node	Any logic element: gate, flip-flop, switch, lamp, clock, or delay.
Port	Named input or output connection point on a node.
Rising edge	Clock transition from 0 to 1. Triggers data capture in flip-flops.
Surface	One of the six primary IDE screens: Project, Design, Verify, Hardware, Export, Import.
Test vector	A truth table row specifying input values and expected output values.
Tick	One discrete simulation time step. All nodes evaluated once per tick in topological order.
Topological sort	Algorithm ensuring nodes are evaluated after their input dependencies.

TERM	DEFINITION
Vivado Kit	The exported ZIP containing `top.vhd`, `top.xdc`, `testbench.vhd`, automation scripts, and documentation files.
XDC	Xilinx Design Constraints file format for pin assignments and I/O standards.

18. Appendices

Appendix A: Logic Primitive Reference

A.1 Basic Logic Gates (2-Input)

PRIMITIVE	INPUTS	OUTPUT	BEHAVIOR
AND	a, b	out	1 when both inputs are 1.
OR	a, b	out	1 when at least one input is 1.
NOT	in	out	Complement of input.
NAND	a, b	out	0 only when both inputs are 1.
XOR	a, b	out	1 when inputs differ.

Note: NOR and XNOR are defined in the type system but are not currently registered in the active component palette. Use NOT+OR or NOT+XOR combinations as equivalents, or use the three-input NOR3 variant which is registered.

A.2 Three-Input Gates

PRIMITIVE	INPUTS	OUTPUT	BEHAVIOR
AND3	a, b, c	out	1 when all three inputs are 1.
OR3	a, b, c	out	1 when any input is 1.
NAND3	a, b, c	out	0 when all three inputs are 1.
NOR3	a, b, c	out	0 when any input is 1.
XOR3	a, b, c	out	1 for odd number of high inputs.

A.3 Sequential Elements

PRIMITIVE	INPUTS	OUTPUTS	BEHAVIOR
D Flip-Flop	D, CLK, EN, RST	Q, $\bar{Q}$	Captures D on rising CLK edge. RST=1 forces Q=0. EN gates capture.
T Flip-Flop	T, CLK, CLR	Q, $\bar{Q}$	T=1 toggles Q on rising edge. T=0 holds.
JK Flip-Flop	J, K, CLK, CLR	Q, $\bar{Q}$	J=1,K=0: set. J=0,K=1: reset. J=K=1: toggle. J=K=0: hold.

A.4 I/O and Signal Elements

PRIMITIVE	PORTS	BEHAVIOR
Switch	out	User-toggled input (0 or 1).
Lamp	in, out	Visual output indicator. Mirrors input.
INPUT	out	External input source.
OUTPUT	in	Terminal output sink.
Power Source	out	Constant 1.
Ground	out	Constant 0.
Clock	out	Periodic oscillation (configurable period).
Delay	in, out	Pass-through with configurable delay (default: 1 tick).
Wire	in, out	Simple pass-through. Output equals input.

Appendix B: Basys 3 Pin Reference

B.1 Slide Switches

SIGNAL	FPGA PIN
SW0	V17
SW1	V16
SW2	W16
SW3	W17
SW4–SW15	See Basys 3 Reference Manual

B.2 LEDs

SIGNAL	FPGA PIN
LD0	U16
LD1	E19
LD2	U19
LD3	V19
LD4–LD15	See Basys 3 Reference Manual

B.3 Push Buttons and Clock

SIGNAL	FPGA PIN	FUNCTION
BTNC	U18	Center button
BTNU	T18	Up button
BTNL	W19	Left button
BTNR	T17	Right button
BTND	U17	Down button
CLK100MHZ	W5	100 MHz on-board oscillator

B.4 Part Number

ATTRIBUTE	VALUE
Device	xc7a35t
Package	cpg236
Speed Grade	-1
Full Part String	`xc7a35t-1cpg236-1`

Appendix C: Generated File Specifications

FILE	ENTITY	KEY PROPERTIES
`top.vhd`	`top`	Synthesizable. Ports match Hardware surface. LVCMOS33 (via XDC). No behavioral process blocks.
`top.xdc`	—	One `set_property PACKAGE_PIN` per port. `CLOCK_BUFFER_TYPE NONE` for switch/button inputs.
`testbench.vhd`	`tb_top`	Simulation only. Exercises all truth table rows. 10 ns clock period for sequential circuits.

Appendix D: Import Fidelity Reference

SOURCE	FIDELITY	CIRCUIT	LAYOUT	VECTORS	PINS
`.rbproj.json` from ZIP	FULL	✓	✓	✓	✓
Structural VHDL	RECONSTRUCTED	✓ (topology)	Auto	—	—
Behavioral VHDL	PARTIAL	Ports only	—	—	—
RedByte `top.vhd`	PARTIAL	Ports only	—	—	—
XDC file	—	—	—	—	✓ (if match)

RedByte

Product Manual · Version 1.0 · March 2026